KOENIG

step forward



Extension Functions

Kotlin for Java Developers

What You Will Learn

1. Extension functions
2. Replacing static utility classes
3. Resolution rules and limitations
4. Nullable receivers
5. Extension properties

The Problem: Static Utility Classes

- **Cannot extend** String, LocalDate or a library class in Java.
- **So we write** XxxUtils with static methods.
- **Call sites** read inside-out and repeat the class name.
- **Poor discovery:** the IDE cannot suggest them on the object.

```java
// Java: a static utility class
public final class ShopUtils {
    private ShopUtils() { }

    public static String slugify(String s) { ... }
    public static String truncate(String s, int max) { ... }
}

// the call site: noisy, nested, reads inside-out
String r = ShopUtils.truncate(
    ShopUtils.slugify(title), 20);
```

Extension Functions: Syntax

- **Receiver type** goes before the function name.
- **Inside,** this is the receiver; this. is optional.
- **Call site** looks like a normal member call.
- **The class** is not modified in any way.

```
fun String.shout(): String = uppercase() + "!"

fun String.truncate(max: Int, ellipsis: String = "..."): String =
    if (length <= max) this else take(max) + ellipsis

fun main() {
    println("kotlin".shout())      // KOTLIN!
    println("kotlin".truncate(3))  // kot...
}
```

fun ReceiverType.name(params): Result

How It Works: A Static Method

```kotlin
// Kotlin (file StringExtensions.kt)
fun String.shout(): String = uppercase() + "!"

// What the compiler generates (as Java):
public final class StringExtensionsKt {
    public static String shout(String $this) {
        return $this.toUpperCase() + "!";
    }
}

// "kotlin".shout()  ==  StringExtensionsKt.shout("kotlin")
```

- **Static method** with the receiver as the first parameter.
- **Class name:** file name plus Kt.
- **No overhead** compared with StringUtils.shout(s).
- **Nothing** is added to String.class.

An extension is syntax sugar for a static helper

Replacing ShopUtils

```java
// Java
String slug = ShopUtils.slugify(
    ShopUtils.capitalize(title));
int n = ShopUtils.wordCount(title);
boolean b = ShopUtils.isBlank(title);
```

```kotlin
// Kotlin
val slug = title.capitalizeFirst().slugify()
val n = title.wordCount()
val b = title.isBlank()      // stdlib extension

// left to right, discoverable with Ctrl+Space,
// null-safety visible in the type
```

Demo: Extensions Instead of ShopUtils

What we build

- **shout, truncate:** member-like calls.
- **javap:** a static method underneath.
- **slugify chain:** left to right.

How the concept helps

- **Readable:** no inside-out calls.
- **Discoverable:** type a dot.
- **Same result:** as the Java class.

Resolution Rule 1: Static Dispatch

```kotlin
open class Payment
class UpiPayment : Payment()

fun Payment.describe() = "a payment"
fun UpiPayment.describe() = "a UPI payment"

fun main() {
    val p: Payment = UpiPayment()
    println(p.describe())            // a payment
    println(UpiPayment().describe())  // a UPI payment
}
```

Member function

Virtual: runtime type decides

Extension function

Static: declared type decides

No overriding for extensions: they are not polymorphic

Resolution Rule 2: Member Wins

```
class Greeter {
    fun greet() = "member"
}

fun Greeter.greet() = "extension"    // warning: shadowed
fun Greeter.greet(n: String) = "hi $n"    // OK: overload

fun main() {
    println(Greeter().greet())         // member
    println(Greeter().greet("Ann"))    // hi Ann
}
```

- **Same signature:** the member always wins.
- **The compiler** warns: extension is shadowed.
- **Different signature** (overload) is allowed.
- **Therefore:** adding a method to a class can silently change which code runs.

Order: member, then local, then extensions by scope

Limitations of Extensions

```
class Account(private val balance: Int)

// fun Account.show() = balance
// error: cannot access 'val balance: Int': it is private

fun Account.describe() = "Account"   // public API only

// val Account.history = mutableListOf<Int>()
// error: extension property cannot be initialized
```

No state

No fields, so no backing field.

Public API only

private and protected are invisible.

Cannot override

Members of the class win.

If you need state or access to private data, use a member

Scope, and Imports

```kotlin
// 1. same package or explicit import
import extensions.slugify
import extensions.*   // whole package

// 2. private to the file
private fun String.internalHelper() = trim()

// 3. member extension: only inside its class
class Report {
    fun String.bold() = "**$this**"
    fun render(t: String) = t.bold()
}
```

- **Visible only** when imported or in the same package.
- **private** keeps a helper inside the file.
- **Member extension:** declared inside a class.

Group related extensions in one file, named after the receiver

Demo: Resolution Rules and Limits

What we build

- **Payment:** declared type decides.
- **Greeter:** the member wins.
- **Account:** no private access, no state.

How the concept helps

- **Predictable:** static dispatch.
- **Safe:** a member is never replaced.
- **Scope:** private and member extensions.

Extensions on Nullable Receivers

```
fun String?.orDash(): String =
    if (this.isNullOrBlank()) "-" else this

fun main() {
    val note: String? = null
    println(note.orDash())      // -   (no ?. needed)
    println("hi".orDash())      // hi

    // stdlib does the same: String?.isNullOrBlank(),
    // String?.orEmpty(), Any?.toString()
}
```

- **Receiver type** String? allows null.
- **Inside,** this may be null: check it.
- **Caller** needs no ?. at the call site.

isNullOrBlank(), orEmpty(), toString() are all of this kind

Nullable: Member vs Extension

```kotlin
val s: String? = null

// member function: needs a safe call
s?.length            // null

// extension on String?: this can be null inside
s.orEmpty().length   // 0
s.isNullOrBlank()    // true

// not allowed: calling a String (non-null) extension
// s.slugify()       // error: only safe (?.) calls
```

Java
s.length() on null: NPE

Kotlin member
needs s?.length

Kotlin extension on String?
s.orEmpty() is safe

Make null handling part of the function, not of the caller

Extension Properties

- **Property** with a custom getter: no backing field.
- **Read-only (val)** or var with get and set.
- **Good for derived values: isWeekend.**

```
import java.time.LocalDate

val LocalDate.isWeekend: Boolean
    get() = dayOfWeek.value >= 6


fun main() {
    println(LocalDate.of(2026, 10, 24).isWeekend)  // true
}
```

Computed on every access: keep getters cheap

Demo: Nullable Receivers & Extension Properties

What we build

- **orDash():** an extension on String?.
- **slugify on String?:** refused.
- **isWeekend:** on LocalDate.

How the concept helps

- **No ?. needed:** the function handles null.
- **Derived values:** read like fields.



Happy Learning :)